Università degli Studi di Catania

Dipartimento di Matematica e Informatica

Corso di Artificial and Swarm Intelligence
Relazione Progetto

Warehouse Swarm Intelligence System

Simulazione Multi-Agente per il Recupero di Oggetti
in Ambienti di Magazzino

Autore:
Giuseppe Bellamacina
Matricola: 1000030349

Professore del corso:
Mario Francesco Pavone

Anno Accademico 2025/2026

Indice

1 Struttura Comune: BaseAgent **3**
1.1 Percezione e Mappa Locale . . . 3
1.1.1 Vision Explored . . . 3
1.1.2 Tombstone per Oggetti Stale . . . 3
1.2 Integrazione della Mappa Condivisa . . . 4
1.3 Protocollo ClearWay . . . 4
1.3.1 Yield Cooldown . . . 4
1.4 Navigazione nei Magazzini: Direzionalità delle Porte . . . 4
1.5 Rilevamento dello Stallo . . . 4

2 Scout Agent **6**
2.1 Ciclo Decisionale . . . 6
2.2 Selezione delle Frontiere . . . 6
2.2.1 Funzione di Utilità con Distanza Sub-lineare . . . 6
2.2.2 Blocco del Target (Target Lock) . . . 7
2.2.3 Blacklist dei Target Recenti . . . 7
2.2.4 Preferenza per Frontiere Lontane . . . 7
2.3 Anti-Clustering . . . 7
2.4 Pattugliamento Ciclico (Stale Coverage) . . . 8
2.5 Relay Passivo e Ricerca del Coordinator . . . 8
2.6 Finestra di Rilevamento Stallo Raddoppiata . . . 8

3 Coordinator Agent **9**
3.1 Raccolta delle Informazioni . . . 9
3.2 Logica di Assegnazione . . . 9
3.3 Strategia di Movimento . . . 10
3.3.1 Riposizionamento verso il Centroide . . . 10
3.3.2 Pattugliamento Anti-Inerzia (Boredom Patrol) . . . 10
3.3.3 Ricerca Attiva dei Retriever (Seek-Retriever) . . . 10
3.4 Consapevolezza Spaziale . . . 10
3.4.1 Rilevamento Chokepoint . . . 10
3.4.2 Rilevamento Blocco di Agenti Vicini . . . 11
3.5 Esplorazione in Modalità map_known . . . 11
3.6 Sincronizzazione tra Coordinator Multipli . . . 11

4 Retriever Agent **12**
4.1 Gerarchia delle Priorità . . . 12
4.2 Auto-Assegnazione con Quattro Livelli di Sicurezza . . . 13
4.3 Cancellazione Reattiva dei Task . . . 13

4.4 Sequenza di Consegna al Magazzino 13
4.5 Selezione del Magazzino Ottimale 14
4.5.1 Ri-routing per Congestione 14
4.5.2 Ordinamento delle Celle Interne (Queueing) 14
4.6 Coordinamento tra Retriever (Peer Negotiation) 14
4.6.1 Broadcast Peer-to-Peer degli Oggetti Scoperti 15
4.6.2 Relay delle Posizioni 15
4.7 Tolleranza ai Guasti: Cargo Drop 15

5 Benchmarks **16**
5.1 Preset A 16
5.2 Preset B 19

Capitolo 1

Struttura Comune: BaseAgent

Tutti e tre i tipi di agente ereditano da `BaseAgent`, che implementa le funzionalità condivise: percezione dell'ambiente, ricezione e invio messaggi, movimento tramite A*, gestione dell'energia e rilevamento dello stato di stallo.

1.1 Percezione e Mappa Locale

Ogni agente mantiene una *mappa locale* rappresentata come matrice numpy, inizialmente tutta `UNKNOWN`. Ad ogni passo, la fase `STAGE_SENSE` aggiorna le celle entro il raggio di visione con il tipo reale della griglia, applicando la logica di *ray casting*: le celle dietro un ostacolo non vengono aggiornate anche se geometricamente entro il raggio, simulando la visibilità lineare degli agenti fisici. Gli oggetti nella zona di visione vengono aggiunti alla lista `known_objects`; le celle magazzino vengono aggiunte a `known_warehouses`.

1.1.1 Vision Explored

Oltre alla mappa locale, ogni agente mantiene una matrice separata `vision_explored` di dimensione `grid_height` $\times$ `grid_width`, inizializzata a zero. Una cella viene marcata a 1 esclusivamente quando entra nel cono di visione diretto dell'agente (ray casting), indipendentemente dalle informazioni ricevute via `MapDataMessage`. Questa distinzione è fondamentale nella modalità `map_known`: quando la topologia della griglia è pre-caricata, `local_map` non contiene celle `UNKNOWN` e la ricerca di frontiere classica fallisce. In questo caso tutti gli agenti usano `vision_explored == 0` come maschera di inesplorato, trattando le celle non ancora scansionate visivamente come frontiere da esplorare. Ciò garantisce che gli oggetti sparsi sulla griglia vengano effettivamente individuati, anche se la topologia è già nota.

1.1.2 Tombstone per Oggetti Stale

Quando un agente scansiona direttamente una cella che in precedenza conteneva un oggetto e la trova vuota (oggetto già raccolto), registra un *tombstone* in `known_objects_cleared` con il passo corrente. Qualsiasi messaggio ricevuto in seguito che affermi la presenza dell'oggetto in quella cella viene rifiutato se il suo timestamp è anteriore al tombstone. Questo impedisce la re-inserzione di oggetti fantasma nella mappa condivisa.

Inoltre, prima di trasmettere un `MapDataMessage`, l'agente esegue uno *scrubbing* preventivo: ogni cella con tipo `OBJECT` che non corrisponde a un oggetto realmente presente sulla griglia viene declassata a `FREE` nel messaggio e nella propria mappa locale. Questo interrompe la catena di ritrasmissione di informazioni obsolete attraverso lo sciame.

1.2 Integrazione della Mappa Condivisa

Alla ricezione di un `MapDataMessage`, l'agente aggiorna le celle della propria mappa ancora `UNKNOWN` con i dati del mittente. Il merge estrae anche le celle magazzino, popolando `known_warehouses`: in questo modo i Retriever scoprono le stazioni di ricarica ben prima di visitarle, purché uno Scout o un Coordinator vicino le abbia già esplorate.

1.3 Protocollo ClearWay

Quando un agente calcola un percorso A* verso il proprio obiettivo ma la cella successiva è occupata da un altro agente fermo, anziché attendere o ricalcolare un percorso alternativo, invia un `ClearWayMessage` direttamente all'occupante. L'agente ricevente, se libero da vincoli immediati, si sposta sulla cella adiacente più vicina libera. Questa negoziazione è ricorsiva (`depth` nel messaggio) fino a una profondità massima: se la catena di spostamenti non si risolve, l'agente iniziante rilascia il messaggio e lascia che A* trovi una via alternativa al passo successivo. Il meccanismo riduce significativamente i blocchi in corridoi stretti senza richiedere alcuna struttura di prenotazione globale.

1.3.1 Yield Cooldown

Dopo aver ceduto una cella in risposta a un `ClearWayMessage`, l'agente registra la posizione ceduta in un dizionario `_yield_cooldown` con un timer di 3 passi. Durante questo periodo, il pathfinder A* penalizza fortemente il passaggio attraverso quella cella, impedendo all'agente di tornare immediatamente sulla posizione appena liberata. Senza questo meccanismo, l'agente cederebbe la cella e nel passo successivo vi si dirigerebbe di nuovo, causando un'oscillazione infinita con il richiedente.

1.4 Navigazione nei Magazzini: Direzionalità delle Porte

La navigazione A* applica un sistema di *forbidden types* contestuale per garantire la direzionalità delle porte del magazzino:

- Un agente **all'interno** del magazzino che si dirige verso l'uscita proibisce le celle `ENTRANCE` come nodi intermedi, impedendo che il percorso passi dalla porta di ingresso.
- Un agente **all'esterno** che si dirige verso l'ingresso proibisce le celle `EXIT` come nodi intermedi, impedendo scorciatoie attraverso l'uscita.
- Un agente in navigazione esterna generica proibisce **tutte** le celle porta (sia ingresso che uscita) come nodi intermedi, evitando che il percorso attraversi i magazzini come scorciatoia.

La cella obiettivo è sempre esclusa dal set di proibizioni, garantendo che l'agente possa effettivamente raggiungere un ingresso o un'uscita quando è la sua destinazione.

1.5 Rilevamento dello Stallo

L'agente registra lo storico delle posizioni recenti. Se per un certo numero di passi consecutivi (`stuck_threshold`) la posizione non cambia, il campo `stuck_counter` viene incrementato.

Superata la soglia, l'agente esegue una *stuck recovery*: invalida il percorso corrente, aggiunge le posizioni degli agenti vicini come evitate e ricalcola un percorso alternativo verso una destinazione leggermente ruotata. Se lo stallo persiste, l'agente si sposta temporaneamente verso una cella casuale libera per sbloccarsi.

Quando il rilevatore di loop o di stallo abbandona un target, la posizione viene inserita nella blacklist `unreachable_targets` con il passo corrente: in questo modo il ciclo decisionale dell'agente non riassegna immediatamente lo stesso obiettivo irraggiungibile nel passo successivo.

Capitolo 2

Scout Agent

Lo Scout è ottimizzato per la copertura massima della griglia nel minor numero di passi. Non recupera oggetti e non riceve compiti: la sua unica responsabilità è esplorare e condividere informazioni.

2.1 Ciclo Decisionale

La priorità dello Scout è deterministica e segue questa sequenza:

1. **Ricarica energetica**: Se l'energia scende sotto la soglia critica (calcolata come la distanza stimata al magazzino più vicino moltiplicata per un fattore di sicurezza), lo Scout interrompe l'esplorazione e si dirige al magazzino. Durante la ricarica, continua a comunicare la mappa locale e gli oggetti scoperti.

2. **Consegna scoperte al Coordinator**: Se lo Scout ha scoperto nuovi oggetti e un Coordinator si trova nel raggio di comunicazione, li trasmette immediatamente tramite `ObjectSpottedMessage`. Se nessun Coordinator è a portata, lo Scout si dirige verso l'ultima posizione nota del Coordinator (`last_seen_coordinator_pos`) per consegnare le informazioni al più presto. Se è trascorso troppo tempo senza trovarlo, viene effettuata una scansione estesa fino a $4\times$ il raggio di comunicazione per localizzarlo.

3. **Selezione frontiera**: Se non è in ricarica, `FrontierExplorer` calcola le frontiere disponibili dalla mappa locale e seleziona quella con utilità massima. La funzione $U(f)$ favorisce cluster grandi e vicini, penalizzando cluster già puntati da altri Scout (comunicazione implicita via `target_position` osservata nei messaggi di stato).

4. **Movimento**: A* calcola il percorso ottimale verso la frontiera selezionata, con le posizioni degli altri agenti come penalizzazione soft.

5. **Comunicazione**: Ad ogni passo, lo Scout condivide con tutti gli agenti entro il raggio `comm_radius` la propria mappa locale aggiornata tramite `MapDataMessage` e la lista degli oggetti scoperti tramite `ObjectSpottedMessage` indirizzati ai Coordinator vicini.

2.2 Selezione delle Frontiere

2.2.1 Funzione di Utilità con Distanza Sub-lineare

La funzione di utilità per la selezione delle frontiere utilizza una distanza con esponente sub-lineare:

$$U(f) = \frac{\text{cluster_size}}{d(f)^{0.4} + 1} - 0{,}5 \cdot n_{\text{agenti}}(f)$$

L'uso di $d^{0.4}$ anziché d lineare impedisce che cluster piccoli ma vicini prevalgano sistematicamente su cluster grandi e distanti. Senza questa correzione, un cluster di 3 celle a distanza 2 vincerebbe contro un cluster di 50 celle a distanza 25, portando a una copertura frammentata.

2.2.2 Blocco del Target (Target Lock)

Quando una frontiera viene selezionata, lo Scout imposta un *target lock* per un numero configurabile di passi (`_TARGET_LOCK_DURATION`). Durante questo periodo, la rivalutazione completa delle frontiere è sospesa anche se ne compaiono di nuove. Il lock viene rilasciato anticipatamente se il target è raggiunto, se il rilevatore di stallo si attiva, o se la durata scade. Questo meccanismo previene cambi di direzione erratici quando lo Scout entra in zone con molti cluster di frontiera adiacenti.

2.2.3 Blacklist dei Target Recenti

Dopo aver raggiunto una frontiera, la sua posizione viene inserita in una blacklist per `_RECENT_TARGET_TTL` passi (default 200). Ciò previene l'oscillazione immediata tra cluster appena esplorati: senza la blacklist, un cluster marginalmente non coperto verrebbe continuamente riselezionato prima di essere completamente scansionato.

2.2.4 Preferenza per Frontiere Lontane

Quando abilitata (`_FAR_FRONTIER_ENABLED`), la selezione preferisce frontiere che richiedono uno spostamento superiore a 2× il raggio di visione (distanza minima configurabile, default 8 celle). Questo spinge lo Scout verso territorio genuinamente inesplorato, evitando di micro-esplorare artefatti di confine tra cluster adiacenti già parzialmente coperti. Se non esistono frontiere lontane, si ricade sulle frontiere disponibili. In modalità `map_known`, questa euristica viene disattivata: le frontiere vicine al confine tra zone scansionate e non scansionate sono le più rilevanti.

2.3 Anti-Clustering

L'anti-clustering è implementato in due livelli. Il primo è un filtro hard: le frontiere che si trovano a meno di `_ANTI_CLUSTER_DIST` celle (default 8) da un altro Scout (visibile nel raggio di comunicazione) vengono escluse completamente dall'insieme dei candidati. Il secondo è una penalizzazione soft nella funzione di utilità:

$$U(f) = \frac{\text{cluster_size}}{d(f)^{0.4} + 1} - 0{,}5 \cdot n_{\text{agenti}}(f)$$

dove $n_{\text{agenti}}(f)$ è il numero di agenti entro 10 celle dalla frontiera f. Se il filtro hard elimina tutte le frontiere (caso raro, ad esempio con molti Scout su una griglia piccola), si ricade sull'insieme completo applicando solo la penalizzazione soft. Il risultato pratico è una divisione organica del territorio senza alcuna coordinazione esplicita.

2.4 Pattugliamento Ciclico (Stale Coverage)

Ogni Scout mantiene una matrice privata `_coverage_step` che registra il passo del modello in cui ciascuna cella è stata l'ultima volta nel raggio di visione. Questa matrice non viene mai trasmessa.

Quando non esistono frontiere inesplorate, lo Scout entra in modalità di *stale coverage patrol*: le celle non visitate per un numero configurabile di passi (`_RESCAN_AGE`) diventano candidati per la ri-esplorazione. La selezione combina età e prossimità:

$$\text{score}(c) = \frac{\text{age}(c)}{d(c) + 1}$$

I primi 50 candidati per punteggio vengono valutati; il primo raggiungibile (cella walkable) viene selezionato come target. In modalità `map_known`, le celle con `vision_explored == 0` hanno priorità assoluta rispetto al ri-pattugliamento di celle già scansionate.

2.5 Relay Passivo e Ricerca del Coordinator

Quando lo Scout ha oggetti scoperti da consegnare, valuta se la catena di relay passivo (via `MapDataMessage` trasmesso ad agenti vicini) è sufficiente. Se il contatto con un qualsiasi agente è avvenuto di recente (`_last_agent_contact_step`), lo Scout presume che le informazioni stiano già propagandosi e continua a esplorare anziché cercare attivamente il Coordinator.

Se il contatto è stale, lo Scout si dirige verso l'ultima posizione nota del Coordinator. All'arrivo, se il Coordinator non è presente, esegue una *wide scan* fino a 4× il raggio di comunicazione per localizzarlo senza navigazione cieca.

2.6 Finestra di Rilevamento Stallo Raddoppiata

A differenza degli altri agenti (che usano uno storico di 15 posizioni), lo Scout utilizza `max_history_length = 30`. Poiché gli Scout si muovono a 2× la velocità degli altri agenti, 15 posizioni coprono solo ∼7 passi reali. Il raddoppio garantisce che la finestra di rilevamento loop copra ∼15 passi effettivi, evitando falsi positivi.

Capitolo 3

Coordinator Agent

Il Coordinator è l'unico agente con visione globale *aggregata*: non può vedere tutta la griglia, ma riceve informazioni da tutti gli Scout vicini e costruisce un modello del mondo più completo rispetto ai Retriever. Il suo ruolo è pianificare chi deve recuperare cosa, evitando ridondanze e massimizzando l'efficienza del sistema.

3.1 Raccolta delle Informazioni

Ogni volta che riceve un `ObjectSpottedMessage` da uno Scout, il Coordinator aggiorna il proprio `known_objects` con la posizione e il valore dell'oggetto. Ogni `TaskStatusMessage` ricevuto da un Retriever aggiorna uno dei dizionari interni: `retriever_carrying` (numero di oggetti trasportati), `retriever_energy` (energia residua) e `objects_being_collected` (set degli oggetti già in recupero).

Il Coordinator estrae la coda di task dichiarata esplicitamente da ciascun Retriever nel `TaskStatusMessage`. Gli slot disponibili per nuove assegnazioni vengono calcolati come:

$$\text{available_slots}(r) = \text{capacity}(r) - |\text{task_queue}(r)| - \text{carrying}(r)$$

Questo approccio, basato sulla dichiarazione autoritativa del Retriever anziché su stime indirette, elimina le race condition tra assegnazione e auto-assegnazione.

3.2 Logica di Assegnazione

La funzione `_plan_task_assignments` calcola per ogni coppia (Retriever, oggetto) una priorità greedy:

$$\text{priority}(o, r) = \frac{\text{value}(o)}{d(r, o) + 1} + b_{\text{spotted}}$$

dove $b_{\text{spotted}} = 0{,}5$ se il Retriever ha già visto l'oggetto direttamente (campo `_spotted_by`), altrimenti 0. I candidati vengono ordinati per priorità decrescente e assegnati in modo greedy rispettando i slot liberi di ciascun Retriever. Al momento dell'invio, il Coordinator trasmette sia il `TaskAssignmentMessage` che un `MapDataMessage` con le celle magazzino note, accelerando la scoperta delle stazioni di deposito da parte dei Retriever che non le hanno ancora esplorate.

Il set `objects_being_collected` è la protezione principale contro la doppia assegnazione: qualsiasi oggetto già assegnato o dichiarato in recupero da un Retriever viene escluso dalla lista di candidati per le nuove assegnazioni.

3.3 Strategia di Movimento

3.3.1 Riposizionamento verso il Centroide

Quando il Coordinator non ha compiti da assegnare, il suo obiettivo di movimento è rimanere raggiungibile dalla propria flotta. All'avvio, finché non riceve nessuna posizione recente da un Retriever, il Coordinator percorre un circuito di cinque waypoint strategici (centro e quattro quadranti della griglia, agganciati alla cella libera più vicina) per coprire tutta la griglia e ristabilire il contatto nel minor numero di passi.

Non appena arrivano posizioni fresche (più recenti di `_POS_MAX_AGE` passi, default 25), calcola il centroide medio di tutti i Retriever e, se la propria distanza da quel punto supera una soglia (pari a $\frac{1}{4}$ del raggio di comunicazione), si sposta verso la cella libera più vicina al centroide.

Il centroide può essere opzionalmente biased verso le posizioni degli oggetti pendenti tramite il parametro `_OBJECT_BIASED_CENTROID`:

$$\text{centroid}_{\text{biased}} = \text{centroid}_{\text{ret}} \cdot (1 - b) + \text{centroid}_{\text{obj}} \cdot b$$

dove $b = 0{,}05$ (default). Questo attira leggermente il Coordinator verso le zone ad alta densità di oggetti senza allontanarlo eccessivamente dai Retriever.

Se la distanza è già entro la soglia, il Coordinator rimane fermo (`IDLE`): non esegue nè esplorazione frontier-based nè passeggiate casuali, poichè il suo compito non è esplorare ma coordinarsi. Questa scelta riduce il consumo energetico e evita che l'agente si allontani involontariamente dagli agenti che gestisce.

3.3.2 Pattugliamento Anti-Inerzia (Boredom Patrol)

Se il Coordinator rimane inattivo per più di `_BOREDOM_THRESHOLD` passi (default 50) senza ricevere nuovi target di esplorazione, forza un ciclo di pattugliamento attraverso i waypoint strategici. Questo impedisce l'accumulo di energia sprecata in attesa passiva quando i task sono pochi e il centroide è già raggiunto.

3.3.3 Ricerca Attiva dei Retriever (Seek-Retriever)

Se esistono oggetti noti ma nessun Retriever è nel raggio di comunicazione, il Coordinator entra in modalità **SEEK-RETRIEVER**: si dirige verso l'ultima posizione dichiarata di un Retriever noto, con l'obiettivo di ristabilire il canale di comunicazione e procedere con l'assegnazione appena possibile. Solo le posizioni più recenti di `_POS_MAX_AGE` passi vengono considerate; le posizioni stale vengono ignorate per evitare di inseguire fantasmi. Se tutte le posizioni note sono stale o già visitate senza successo, il Coordinator torna al pattugliamento.

3.4 Consapevolezza Spaziale

3.4.1 Rilevamento Chokepoint

Il Coordinator valuta se la propria posizione corrente è un *chokepoint*: una cella con al massimo 2 vicini walkable. In tal caso, si sposta per evitare di bloccare il passaggio dei Retriever in corridoi stretti.

3.4.2 Rilevamento Blocco di Agenti Vicini

Se un agente adiacente ha `stuck_counter` $\geq$ `2` (bloccato da almeno due passi consecutivi), il Coordinator rileva di essere potenzialmente la causa del blocco e si sposta per liberare il passaggio. Questa euristica reattiva è complementare al protocollo ClearWay.

3.5 Esplorazione in Modalità map_known

In modalità `map_known`, il Coordinator utilizza `vision_explored == 0` per identificare le celle non ancora scansionate visivamente. Quando ha task pendenti ed è già vicino al centroide, bias l'esplorazione verso le celle ignote più vicine al centroide. In assenza di task, tratta le celle non scansionate come frontiere normali.

3.6 Sincronizzazione tra Coordinator Multipli

In configurazioni con più di un Coordinator, il rischio di doppia assegnazione aumenta. Il `CoordinatorSyncMessage` viene scambiato periodicamente tra Coordinator vicini e contiene la lista delle assegnazioni attive (`assigned_tasks`) e degli oggetti in recupero (`objects_being_collected`). Alla ricezione, ogni Coordinator fonde le informazioni ricevute con le proprie, aggiornando i set interni.

La sincronizzazione è soggetta a rate limiting: un solo messaggio ogni `_SYNC_RATE_LIMIT` passi (default 10) per coppia di Coordinator, controllato tramite timestamp per-Coordinator nel dizionario `last_sync_step`. Questa propagazione non è istantanea — si propaga a onde tramite la comunicazione locale — ma è sufficiente per eliminare la stragrande maggioranza delle doppie assegnazioni in scenari realistici, bilanciando correttezza e overhead comunicativo.

Capitolo 4

Retriever Agent

Il Retriever è l'agente più complesso: deve recuperare oggetti fisicamente sulla griglia, gestire la capacità di carico, navigare verso il magazzino, ricaricarsi e uscire, tutto mantenendo una comunicazione attiva con Coordinator e altri Retriever.

4.1 Gerarchia delle Priorità

Il ciclo decisionale del Retriever è una gerarchia di priorità decrescenti, valutate in sequenza ad ogni passo della fase `STAGE_DECIDE`:

1. **P1 — Consegna**: Se trasporta almeno un oggetto e la capacità è esaurita (oppure la coda è vuota), si dirige al magazzino tramite la sequenza di consegna. In caso di capacità piena, tutti i claim residui nella coda vengono rilasciati immediatamente, rendendoli disponibili per i peer.

2. **P2 — Ricarica d'emergenza**: Se l'energia scende sotto la soglia critica e non sta già consegnando, il Retriever interrompe qualsiasi operazione e si dirige al magazzino per ricaricarsi. La soglia è dinamica: se il magazzino più vicino noto dista d celle, la soglia di allarme è $d \times 1.5$ unità di energia, garantendo sempre un margine di sicurezza.

3. **P3 — Esecuzione compiti assegnati**: Se ha una coda di compiti assegnati dal Coordinator, preleva il prossimo oggetto dalla coda e si dirige verso di esso. La coda viene riordinata per distanza Manhattan dalla posizione corrente ad ogni passo (`_TASK_QUEUE_REORDER`), garantendo che il task più vicino sia sempre il primo.

4. **P3b — Pickup opportunistico**: Dopo aver selezionato il task primario, il Retriever scansiona `known_objects` alla ricerca di oggetti non reclamati entro il raggio di visione. Se la capacità residua lo consente, reclama atomicamente fino a `spare_capacity` oggetti aggiuntivi. Questo massimizza il fattore di utilizzo del carico durante il transito verso il task primario.

5. **P4 — Auto-assegnazione (modalità hive-mind)**: In assenza di compiti assegnati, il Retriever scansiona l'intero `known_objects` accumulato (non solo le celle visibili) e si auto-assegna l'oggetto più conveniente tramite `try_claim_object` atomico. Questo meccanismo elimina i tempi morti tra un'assegnazione e l'altra, sfruttando la conoscenza condivisa tramite le mappe ricevute dagli altri agenti.

6. **P5 — Mini-esplorazione**: Se non ha né compiti né oggetti noti, il Retriever esegue una breve frontier-based exploration locale per scoprire oggetti nell'area circostante. In

modalità `map_known` usa `vision_explored == 0` per identificare celle non scansionate, con anti-clustering rispetto ai Coordinator vicini.

4.2 Auto-Assegnazione con Quattro Livelli di Sicurezza

La funzione `_try_self_assign_visible` è protetta da quattro filtri in cascata per prevenire doppie assegnazioni:

1. **Verifica sulla griglia**: Se l'oggetto non è più presente in `model.grid.objects`, viene scartato.
2. **Età del claim globale**: Se un altro agente ha un claim fresco (meno di `_STALE_CLAIM_AGE` passi, default 50) sull'oggetto, viene saltato.
3. **Controllo code peer**: Se l'oggetto appare nella `task_queue` di un Retriever nelle vicinanze, viene saltato.
4. **Claim atomico**: `CommunicationManager.try_claim_object` garantisce first-come-first-served a livello globale.

I claim più vecchi di `_STALE_CLAIM_AGE` passi sono considerati scaduti e possono essere acquisiti da un altro Retriever. Questo meccanismo di *stale claim takeover* previene che un Retriever fuori energia o bloccato impedisca indefinitamente il recupero di un oggetto.

4.3 Cancellazione Reattiva dei Task

Alla ricezione di un `MapDataMessage`, il Retriever verifica immediatamente se gli oggetti nella propria coda sono ancora presenti sulla griglia. Se un oggetto risulta assente (già raccolto da un peer), il task viene rimosso e il claim rilasciato senza attendere la riassegnazione da parte del Coordinator. Questo riduce i passi sprecati verso oggetti fantasma.

4.4 Sequenza di Consegna al Magazzino

La navigazione al magazzino avviene in quattro sotto-fasi gestite da una macchina a stati interna (`_wh_step`):

1. **Approach**: Il Retriever si dirige verso la cella `WAREHOUSE_ENTRANCE` più vicina del magazzino ottimale (selezionato da `get_best_warehouse_for`). Il percorso A* usa le forbidden types per forzare il passaggio dall'ingresso, escludendo le celle di uscita come nodi intermedi.
2. **Deposit**: Entrato nel magazzino attraverso l'ingresso, il Retriever deposita tutti gli oggetti trasportati, che vengono conteggiati nelle metriche e rimossi dalla griglia. Lo stato passa a `RECHARGE` se l'energia è sotto soglia, altrimenti direttamente a `EXIT`.
3. **Recharge**: Il Retriever rimane sulla cella magazzino mentre viene ricaricato a tasso costante (`recharge_rate` unità per passo) fino al recupero completo dell'energia massima. Durante la ricarica continua a ricevere messaggi.

4. **Exit**: Il Retriever si dirige verso la cella `WAREHOUSE_EXIT` e la attraversa. Uscito dal magazzino, `_wh_step` viene azzerato; prima di tornare all'esplorazione, il Retriever tenta immediatamente una nuova auto-assegnazione tramite `known_objects`, riducendo i passi a vuoto tra una consegna e la successiva.

4.5 Selezione del Magazzino Ottimale

La funzione `get_best_warehouse_for` seleziona la stazione di deposito/ricarica ottimale tra tutte quelle note all'agente. Il punteggio per ogni stazione combina tre fattori:

- **Distanza effettiva A***: La distanza viene calcolata tramite il pathfinder A*, che tiene conto degli ostacoli reali sulla griglia. L'uso della distanza Manhattan causerebbe errori significativi in mappe con corridoi e barriere: un magazzino "vicino" in linea d'aria può richiedere un percorso effettivo doppio o triplo a causa degli ostacoli. Se il percorso non esiste, viene applicata una penalizzazione di +500 sulla distanza Manhattan come fallback.
- **Congestione**: Il numero di Retriever già diretti verso ciascun magazzino viene conteggiato esaminando `_wh_station` di tutti gli agenti. La penalizzazione è lineare: penalty $= 8 \times n_{\text{heading}}$.
- **Fattibilità energetica**: Se la distanza stimata (moltiplicata per un fattore di percorso 1.4 per tenere conto di deviazioni residue) supera l'energia residua dell'agente, la stazione riceve una penalizzazione aggiuntiva di 200 punti, rendendola di fatto non preferibile.

$$\text{score}(s) = d_{\text{A}*}(pos, s) + 8 \cdot n_{\text{heading}}(s) + \begin{cases} 200 & \text{se } d_{\text{A}*} \cdot 1{,}4 > E_{\text{agent}} \\ 0 & \text{altrimenti} \end{cases}$$

4.5.1 Ri-routing per Congestione

Se il Retriever è in fase `approach` da almeno 6 passi e il numero di agenti diretti allo stesso ingresso supera la soglia `_WH_CONGESTION_THRESHOLD`, viene rivalutata la scelta del magazzino escludendo quello corrente. Se esiste un'alternativa con minor congestione, il Retriever cambia rotta immediatamente, azzerando il percorso A* e il contatore di approccio.

4.5.2 Ordinamento delle Celle Interne (Queueing)

Le celle interne al magazzino sono ordinate per distanza dall'ingresso in modo decrescente: la cella più lontana dall'ingresso è lo slot 0 (il più profondo). Gli agenti riempiono il magazzino dalla posizione più interna verso l'ingresso, garantendo che nessun agente si fermi direttamente sulla porta anche in condizioni di alta congestione.

4.6 Coordinamento tra Retriever (Peer Negotiation)

I conflitti tra Retriever che puntano allo stesso oggetto si risolvono in due livelli:

1. **Claim atomico**: Ogni auto-assegnazione passa per `CommunicationManager.try_claim_object`, operazione atomica condivisa da tutti gli agenti. Solo un Retriever ottiene il claim; i contendenti ricevono immediatamente un fallimento e selezionano un altro oggetto.

2. **Peer yield via TaskStatusMessage**: Dopo ogni claim riuscito, il Retriever diffonde un `TaskStatusMessage` a tutti i vicini. Alla ricezione, ogni peer rimuove dalla propria `task_queue` gli oggetti già presenti nella coda del mittente, rilasciando i claim corrispondenti e aggiornando preventivamente la propria pianificazione senza attendere un fallimento esplicito.

L'assenza di un arbitro centralizzato e la natura broadcast del `TaskStatusMessage` garantiscono convergenza rapida: nella pratica, le collisioni residue dopo il claim atomico sono rare e vengono risolte entro uno-due passi di comunicazione.

4.6.1 Broadcast Peer-to-Peer degli Oggetti Scoperti

I Retriever possono comunicare direttamente tra loro tramite `RetrieverEventMessage` con eventi `object_spotted` e `cargo_dropped`. Alla ricezione, il peer aggiunge l'oggetto a `known_objects` se non già noto e non contrassegnato da tombstone. Questo crea catene di scoperta opportunistiche che propagano la conoscenza degli oggetti peer-to-peer, senza intermediazione del Coordinator.

4.6.2 Relay delle Posizioni

Prima di inviare un `TaskStatusMessage`, ogni Retriever aggiorna la propria posizione nel dizionario `retriever_positions`. Questa informazione viene inclusa nei `MapDataMessage` e propagata da tutti i riceventi, creando una rete di relay in cui ogni agente diventa un nodo di trasmissione per le posizioni di tutti i Retriever.

4.7 Tolleranza ai Guasti: Cargo Drop

Se l'energia del Retriever raggiunge zero, tutti gli oggetti trasportati vengono rilasciati sulle celle libere adiacenti in una spirale espansiva. Le posizioni degli oggetti rilasciati vengono trasmesse ai vicini tramite `RetrieverEventMessage` con evento `cargo_dropped`, permettendo ad altri Retriever di recuperarli. Questo meccanismo impedisce che un guasto energetico causi la perdita permanente degli oggetti già raccolti.

Capitolo 5

Benchmarks

I benchmark sono stati eseguiti su due configurazioni di magazzino (Preset A e Preset B) con lo stesso numero di run per garantire significatività statistica. Per ogni preset vengono riportati quattro grafici e una tabella riepilogativa.

5.1 Preset A

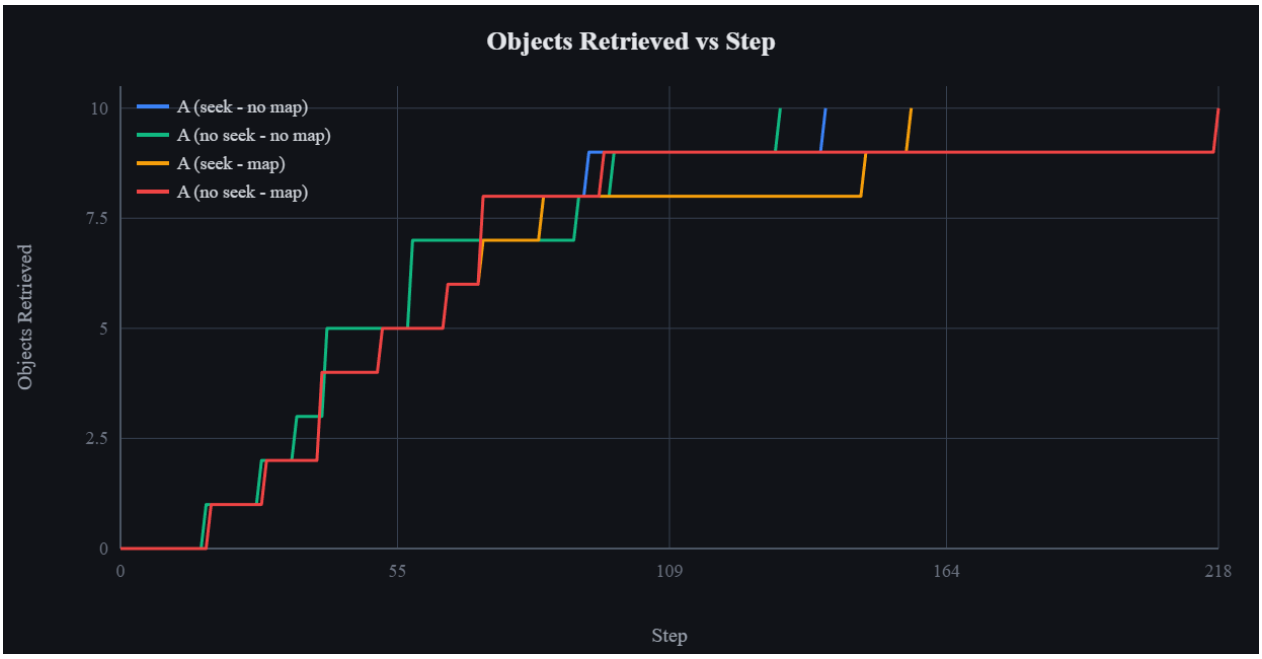

Figura 5.1: Andamento del recupero oggetti — Preset A

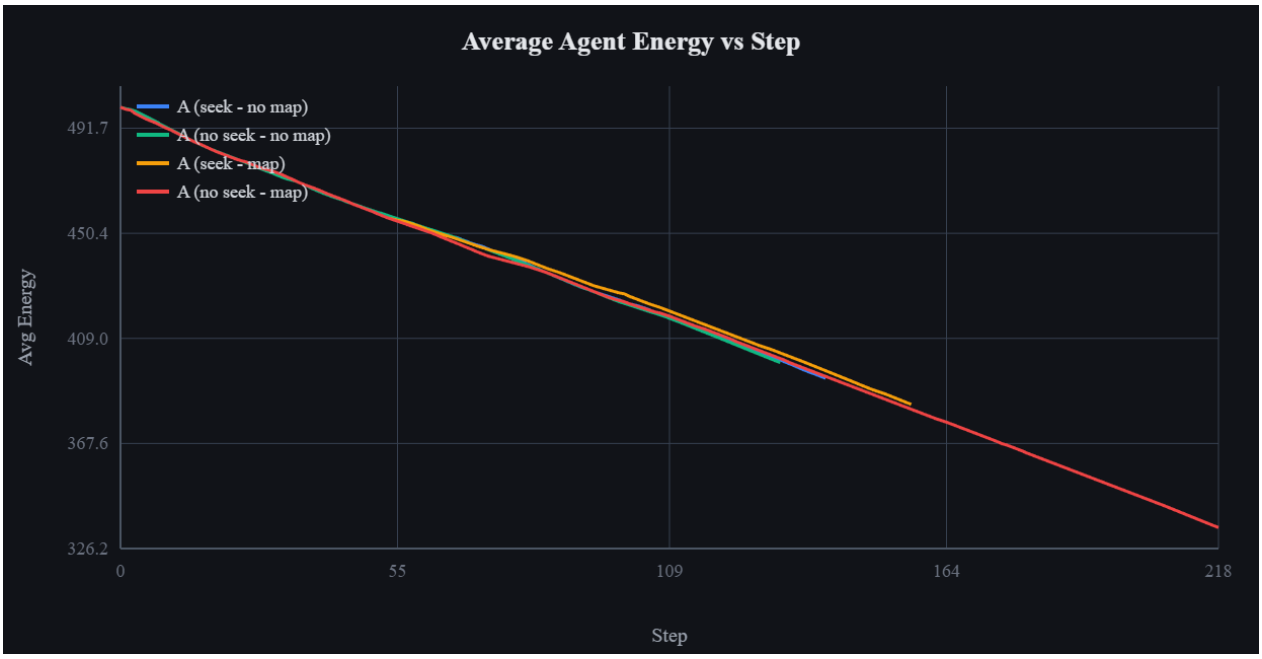

Figura 5.2: Consumo energetico — Preset A

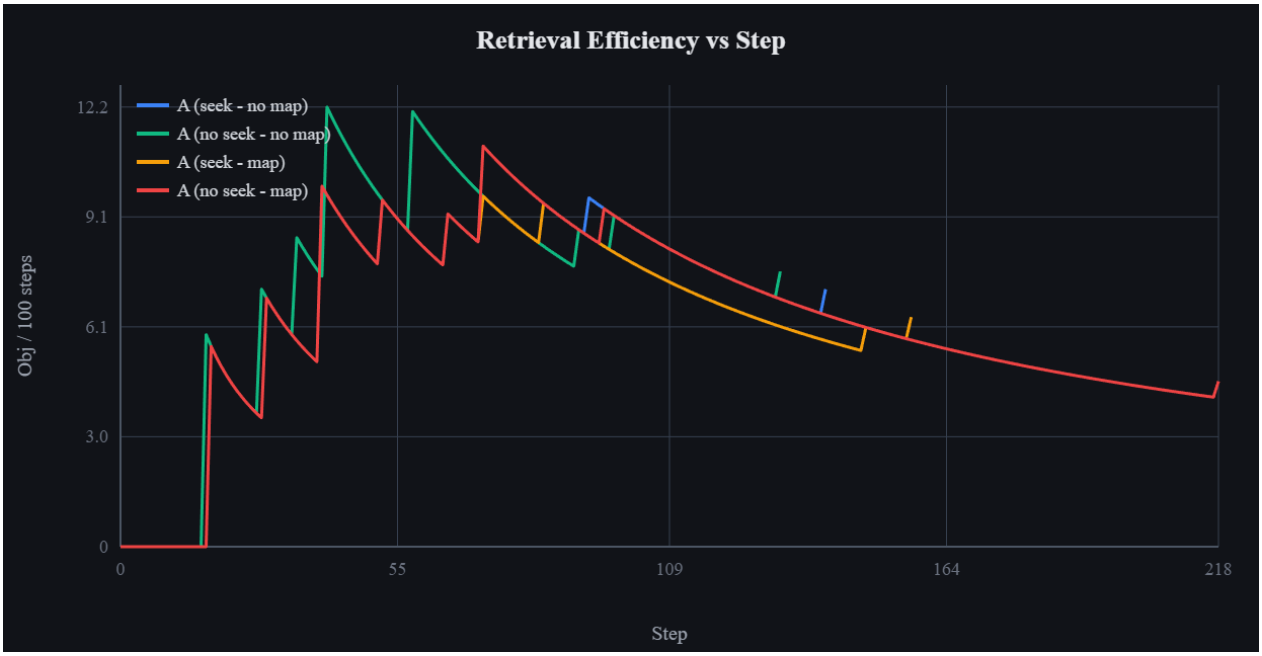

Figura 5.3: Efficienza energetica — Preset A

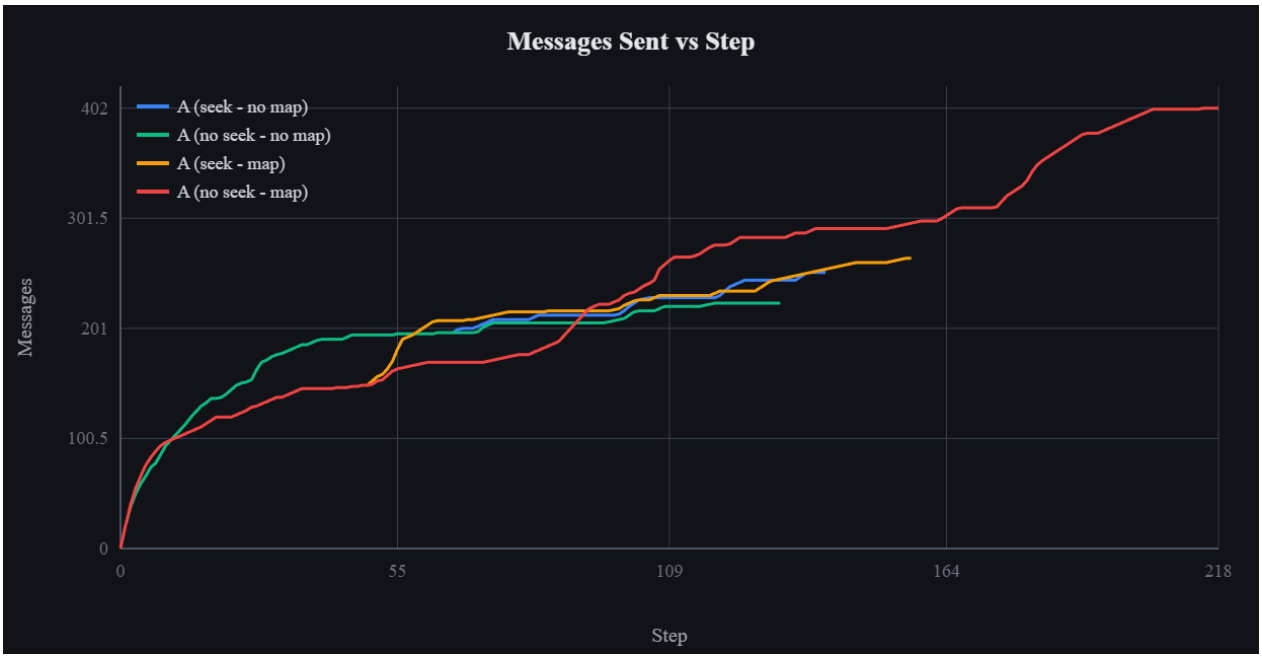

Figura 5.4: Messaggi scambiati — Preset A

Run	Steps	Retrieved	Completion	Efficiency	Avg Energy	Messages	Agents
A (seek - no map)	140	10/10	100.0%	7.14	446	252	5 (1S/1C/3R)
A (no seek - no map)	131	10/10	100.0%	7.63	449	224	5 (1S/1C/3R)
A (seek - map)	157	10/10	100.0%	6.37	440	265	5 (1S/1C/3R)
A (no seek - map)	218	10/10	100.0%	4.59	416	402	5 (1S/1C/3R)

Figura 5.5: Tabella riepilogativa — Preset A

5.2 Preset B

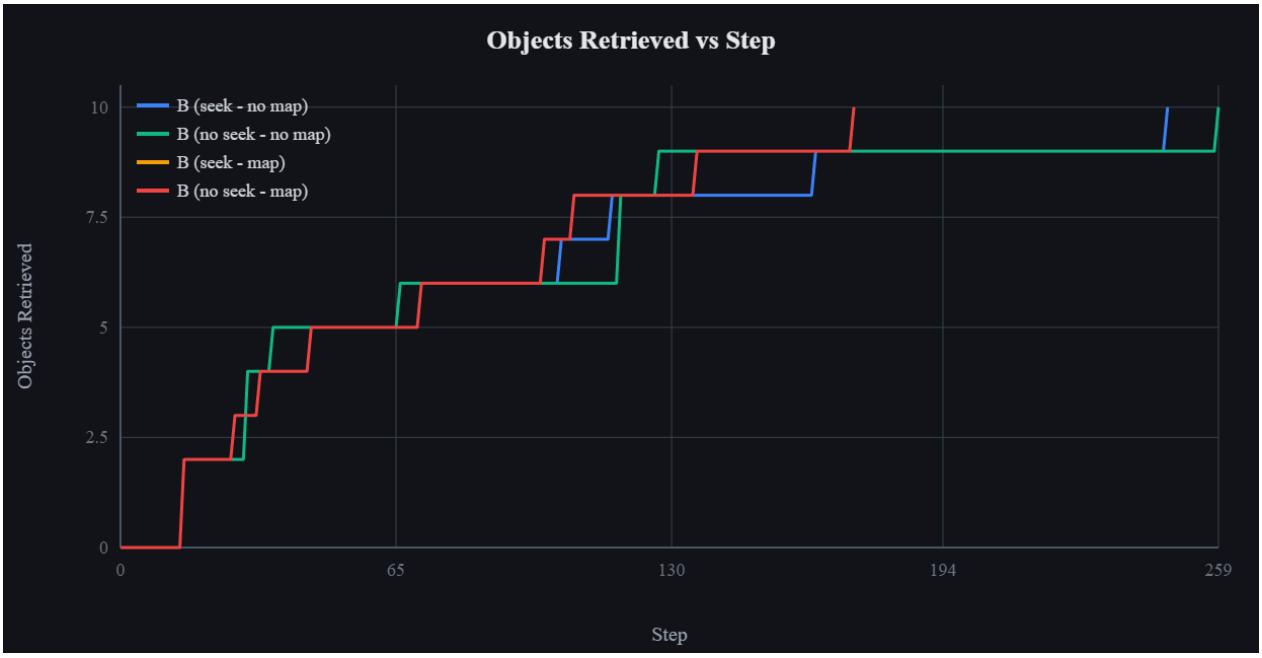

Figura 5.6: Andamento del recupero oggetti — Preset B

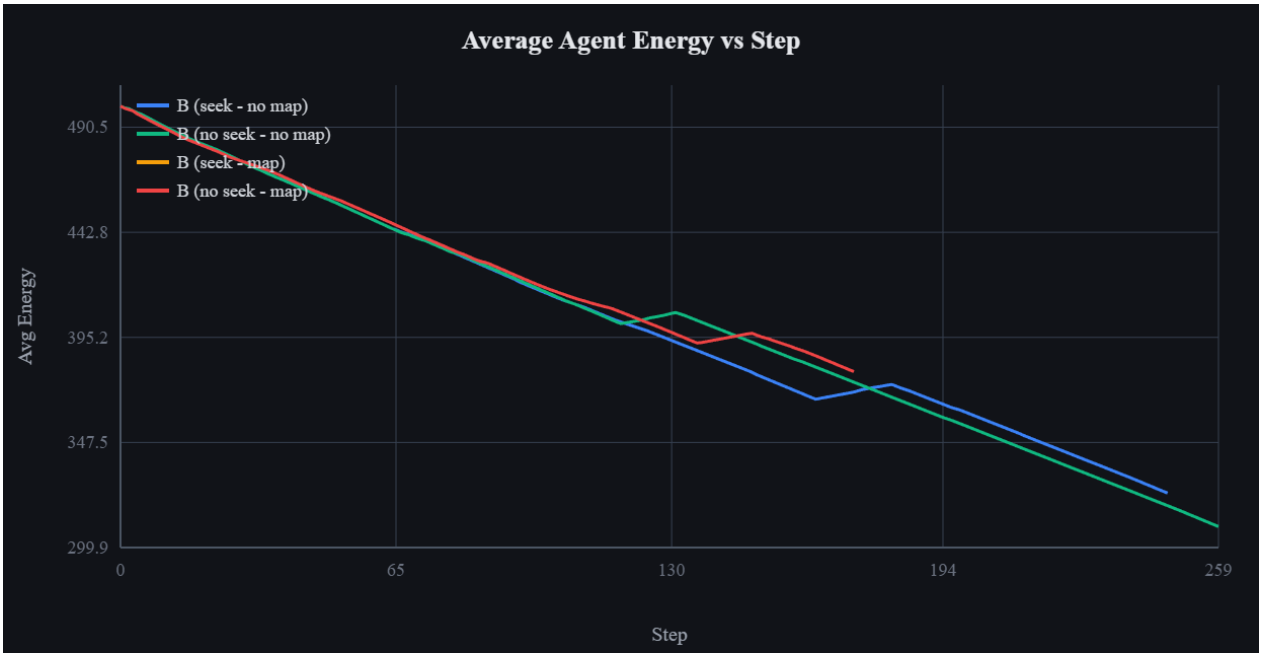

Figura 5.7: Consumo energetico — Preset B

Figura 5.8: Efficienza energetica — Preset B

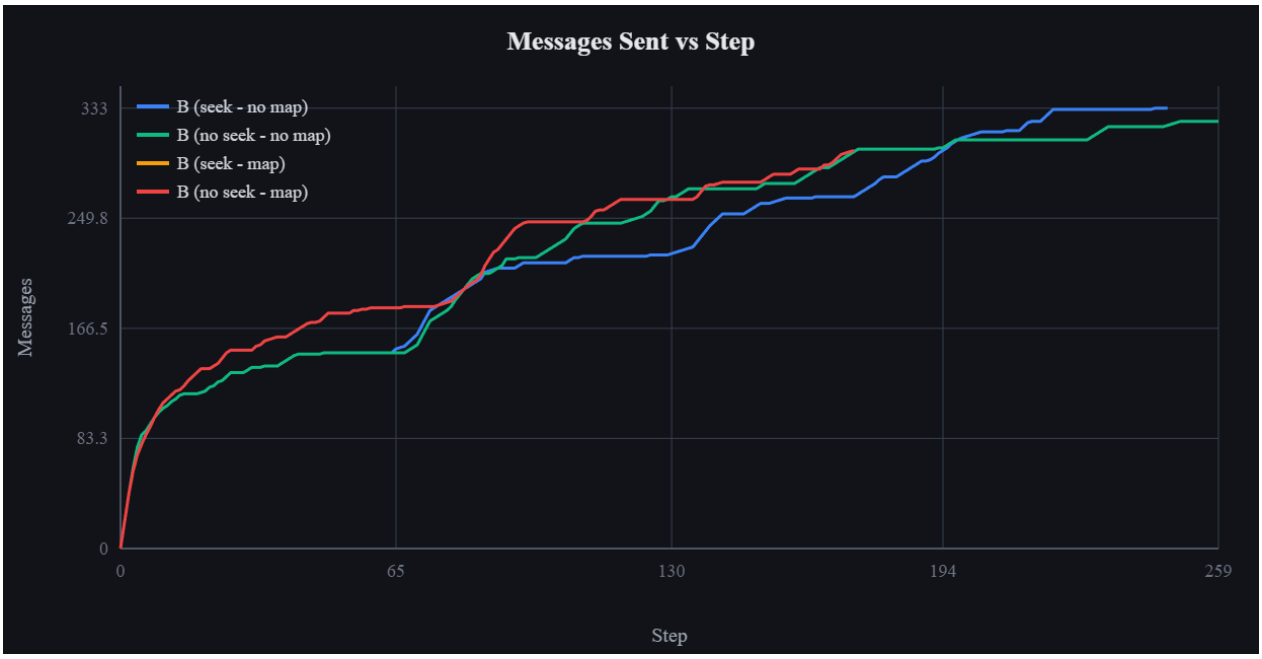

Figura 5.9: Messaggi scambiati — Preset B

Run	Steps	Retrieved	Completion	Efficiency	Avg Energy	Messages	Agents
B (seek - no map)	247	10/10	100.0%	4.05	406	333	5 (1S/1C/3R)
B (no seek - no map)	259	10/10	100.0%	3.86	402	323	5 (1S/1C/3R)
B (seek - map)	173	10/10	100.0%	5.78	433	301	5 (1S/1C/3R)
B (no seek - map)	173	10/10	100.0%	5.78	433	301	5 (1S/1C/3R)

Figura 5.10: Tabella riepilogativa — Preset B